

Layer 1: Math Foundations

Detailed study guide

4 sections • 8 weeks • ~40 topics • exercises throughout

What this document covers

This is a detailed breakdown of the four mathematical topics in Layer 1 of the roadmap. For each topic you will find: what it is, why it matters for AI, exactly what to learn, concrete examples, and exercises to test your understanding.

The four topics are not equally weighted. Linear algebra is the most important by far — it underpins every operation in machine learning. Plan to spend at least half your time there.

Time allocation for Layer 1 (8 weeks)

Linear algebra:	3 weeks	(most important, take your time)
Calculus:	2 weeks	(focus on derivatives and chain rule)
Probability & stats:	2 weeks	(distributions, expectation, variance)
Information theory:	1 week	(entropy and Shannon bounds)

Section 1 Linear Algebra

The language of machine learning

Every operation in a neural network is a matrix multiplication. When an LLM processes a token, it projects it into query/key/value spaces using matrix multiplies. When TurboQuant rotates a vector, that is a matrix multiply. When gradient descent updates weights, it uses vector operations. You cannot avoid linear algebra — it is the language ML is written in.

1.1 Vectors

A vector is just a list of numbers. In ML, vectors represent almost everything: a word, an image, a token, an embedding. Understanding what vectors are and how they behave in space is the starting point.

What to learn

- A vector as a point in space vs a direction and magnitude
- Notation: column vector vs row vector
- Vector addition and scalar multiplication — what they mean geometrically
- The L2 norm (Euclidean length): $\|x\| = \sqrt{x_1^2 + x_2^2 + \dots + x_d^2}$
- Unit vectors — vectors with norm = 1
- Normalizing a vector: $x / \|x\|$

Why it matters

In TurboQuant, all input vectors are normalized to unit norm before quantization. This is not arbitrary — the algorithm's distortion guarantees are proven for unit-norm vectors. The paper writes x in S^{d-1} , meaning x lives on the unit hypersphere. That is just a normalized vector in d dimensions.

Concrete example

A 3D word embedding might be: $x = [0.2, -0.8, 0.5]$. Its norm is $\sqrt{0.04 + 0.64 + 0.25} = \sqrt{0.93} = 0.964$. Normalized: $x / 0.964 = [0.207, -0.830, 0.519]$. Now $\|x_{\text{normalized}}\| = 1.0$.

1.2 The dot product

The dot product is the single most important operation in ML. Attention is built from dot products. Similarity search is dot products. Inner product quantization (TurboQuant's second algorithm) is all about preserving dot products under compression.

What to learn

- Definition: $x \cdot y = x_1 y_1 + x_2 y_2 + \dots + x_d y_d$
- Geometric interpretation: $x \cdot y = \|x\| \cdot \|y\| \cdot \cos(\theta)$
- When the dot product is zero: the vectors are perpendicular (orthogonal)
- When the dot product is maximized: the vectors point in the same direction
- Cosine similarity: $(x \cdot y) / (\|x\| \cdot \|y\|)$ — measures angle regardless of magnitude
- Why unit vectors simplify things: if $\|x\| = \|y\| = 1$, then $x \cdot y = \cos(\theta)$

Why it matters

The TurboQuant inner product quantizer (Algorithm 2) is designed specifically to preserve dot products. Its guarantee is that $E[\langle y, Q^{-1}(Q(x)) \rangle] = \langle y, x \rangle$ — the expected dot product after quantizing and dequantizing x equals the true dot product. This is what 'unbiased inner product estimator' means.

1.3 Matrices

A matrix is a 2D array of numbers. In ML, matrices represent transformations, weight layers, and datasets. Every layer in a neural network applies a matrix multiplication to its input.

What to learn

- Matrix as a transformation: multiplying by M transforms a vector into a new vector
- Matrix multiplication: $(AB)_{ij} = \sum_k A_{ik} \cdot B_{kj}$
- Shapes matter: A ($m \times k$) times B ($k \times n$) gives C ($m \times n$)
- The transpose: swapping rows and columns
- The identity matrix I : $M @ I = M$
- Matrix inverse: $M @ M^{-1} = I$ (not all matrices have inverses)
- Determinant: whether a matrix is invertible and how it scales space

Matrix multiplication in practice (NumPy)

```
import numpy as np
```

```
A = np.array([[1, 2], [3, 4]]) # 2x2 matrix
```

```
x = np.array([1, 0])          # 2D vector
```

```
y = A @ x # matrix-vector multiply -> [1, 3]
```

```
B = A @ A # matrix-matrix multiply
```

```
At = A.T # transpose
```

1.4 Orthogonal matrices

This is the specific type of matrix that TurboQuant uses as its random rotation. An orthogonal matrix Q satisfies $Q^T @ Q = I$. This means: $Q^{-1} = Q^T$, and multiplying by Q preserves all distances and angles.

What to learn

- Definition: $Q^T Q = I$ and $Q Q^T = I$
- Property: $\|Q x\| = \|x\|$ for any vector x (orthogonal matrices preserve norm)
- Property: the dot product is preserved: $(Qx) \cdot (Qy) = x \cdot y$
- What rotation looks like geometrically — spinning the coordinate system
- Why rotation does not change the information content of a vector

Why this is the key insight in TurboQuant

TurboQuant's entire strategy rests on this: multiplying by a random orthogonal matrix Q turns any worst-case input vector into one that behaves like a random point on the unit sphere. This is the 'randomization' step. Because Q preserves norms, the quantized representation can be unrotated at dequant time with no loss.

1.5 QR decomposition

QR decomposition factors a matrix $A = Q R$, where Q is orthogonal and R is upper triangular. To generate a random orthogonal matrix (as TurboQuant does), you generate a random Gaussian matrix and take its Q factor.

What to learn

- What QR decomposition produces and why Q is orthogonal
- How to compute it in NumPy: $Q, R = \text{np.linalg.qr}(A)$
- Why a random Gaussian matrix's Q factor is uniformly random over orthogonal matrices (Haar measure)

TurboQuant's rotation matrix in code

```
import numpy as np

d = 1536 # vector dimension

# Generate a random Gaussian matrix
G = np.random.randn(d, d)

# QR decomposition — Q is a random orthogonal matrix
Q, _ = np.linalg.qr(G)

# Verify it's orthogonal:  $Q^T Q$  should equal identity
assert np.allclose(Q.T @ Q, np.eye(d), atol=1e-5)

# Rotate a vector
```

```
x = np.random.randn(d)
x = x / np.linalg.norm(x) # normalize
y = Q @ x                 # rotate
```

1.6 Eigenvalues and eigenvectors

An eigenvector of a matrix A is a vector that only gets scaled (not rotated) when multiplied by A . The scale factor is the eigenvalue. This is foundational for understanding PCA, covariance matrices, and why certain transformations behave the way they do.

What to learn

- Definition: $A v = \lambda v$, where v is the eigenvector and λ is the eigenvalue
- Geometric meaning: eigenvectors are the 'natural axes' of a transformation
- Eigendecomposition: $A = V \Lambda V^{-1}$
- Symmetric matrices always have real eigenvalues and orthogonal eigenvectors
- PCA as eigendecomposition of the covariance matrix

Exercises for Section 1

#	Exercise
1	Write a function that takes two vectors and returns their dot product. Do not use <code>np.dot</code> — implement it with a loop.
2	Normalize a random 512-dimensional vector to unit norm. Verify that its norm equals 1.0.
3	Generate a random 4x4 matrix. Multiply it by its transpose. What do you notice about the result?
4	Generate a random 64x64 Gaussian matrix. Compute its QR decomposition. Verify $Q^T Q = I$.
5	Write a function that rotates a 2D vector by a given angle using a rotation matrix. Plot several vectors before and after rotation.
6	Take a 512-dim vector, multiply it by a random orthogonal matrix Q , and verify the norm is preserved.
7	Compute the eigenvalues of a 3x3 symmetric matrix using <code>np.linalg.eigh</code> . Verify $A v = \lambda v$ for each eigenpair.

Section 2 Calculus

How to find the minimum of any function

Calculus tells us how functions change. In ML, the core task is: find the weights of a neural network that minimize a loss function. We do this with gradient descent, which requires computing derivatives. If you understand derivatives and the chain rule, you understand backpropagation.

2.1 Derivatives

A derivative measures how fast a function changes. $f'(x)$ = the slope of f at x . If $f'(x) > 0$, f is increasing. If $f'(x) = 0$, you might be at a minimum or maximum.

What to learn

- The limit definition: $f'(x) = \lim_{h \rightarrow 0} [f(x+h) - f(x)] / h$
- Power rule: $d/dx [x^n] = n x^{n-1}$
- Common derivatives: e^x , $\ln(x)$, $\sin(x)$, $\cos(x)$
- The derivative as slope of the tangent line
- Finding minima: set $f'(x) = 0$ and solve

2.2 The chain rule

The chain rule is how we differentiate composite functions. This is the mathematical foundation of backpropagation. If you have $f(g(x))$, the derivative is $f'(g(x)) * g'(x)$. In a neural network with many layers, you apply this rule repeatedly.

What to learn

- Chain rule: $d/dx [f(g(x))] = f'(g(x)) * g'(x)$
- Chain rule with multiple variables
- How backprop is just the chain rule applied to a computational graph
- Worked example: differentiate $\text{sigmoid}(W @ x + b)$ with respect to W

The chain rule is backpropagation

A neural network is a composition of functions: $\text{output} = f_n(f_{n-1}(\dots f_1(x)\dots))$

To update weights, we need: $d(\text{loss})/d(\text{weight})$ for every weight

The chain rule lets us compute this by multiplying local gradients backwards

This is all backpropagation is — the chain rule applied efficiently to a computational graph

2.3 Partial derivatives and gradients

When a function has multiple inputs, the partial derivative with respect to one input measures how the function changes when you vary that input while holding the others fixed. The gradient is the vector of all partial derivatives.

What to learn

- Partial derivative: $d/dx_i [f(x_1, \dots, x_n)]$
- The gradient vector: $\text{grad } f = [df/dx_1, df/dx_2, \dots, df/dx_n]$
- The gradient points in the direction of steepest ascent
- Gradient descent: $x_{\text{new}} = x - \text{learning_rate} * \text{grad } f(x)$
- The Jacobian: the matrix of all partial derivatives for a vector-valued function

2.4 Gradient descent

Gradient descent is how neural networks are trained. You compute the gradient of the loss with respect to all weights, then take a small step in the opposite direction (downhill). Repeat until converged.

What to learn

- The loss landscape — what you are navigating
- Learning rate: too large = overshooting, too small = very slow
- Local vs global minima — and why deep networks often find good solutions anyway
- Mini-batch gradient descent — the practical variant
- Momentum — using past gradients to speed up convergence

Gradient descent in 10 lines of Python

```
def f(x): return x**2 + 3*x + 2 # function to minimize
def grad_f(x): return 2*x + 3 # its gradient (derivative)

x = 10.0 # starting point
lr = 0.1 # learning rate

for step in range(50):
    g = grad_f(x) # compute gradient
    x = x - lr * g # step downhill

print(x) # converges to x = -1.5, the minimum of f
```

Exercises for Section 2

#	Exercise
1	Compute by hand: $d/dx [3x^3 - 2x^2 + 5x - 1]$. Then verify with sympy.
2	Apply the chain rule to differentiate: $f(x) = (x^2 + 1)^3$. Expand and verify.
3	Implement gradient descent from scratch to minimize $f(x) = x^2 - 4x + 4$. Plot x vs step.
4	Compute the gradient of $f(x,y) = x^2 + 2xy + y^2$ at point $(1, 2)$.
5	Implement gradient descent for a 2D function. Visualize the path on a contour plot.
6	Experiment with learning rates: run gradient descent with $lr=0.001, 0.1, 1.0, 2.0$. What happens?

Section 3 Probability & Statistics

Reasoning under uncertainty

ML models are fundamentally probabilistic. They do not output a single answer — they output distributions. They are trained on samples, evaluated on distributions, and their guarantees are stated as expected values. TurboQuant's core theorems are all probabilistic statements about expected distortion.

3.1 Random variables and distributions

What to learn

- Random variable: a variable whose value is determined by a random process
- Probability mass function (PMF): for discrete random variables
- Probability density function (PDF): for continuous random variables
- Cumulative distribution function (CDF): $P(X \leq x)$
- Support: the set of values a random variable can take

Key distributions to know

- Uniform distribution: all values equally likely
- Bernoulli distribution: 0 or 1 with probability p
- Binomial distribution: number of successes in n Bernoulli trials
- Gaussian (normal) distribution: the bell curve — appears everywhere due to the CLT
- Beta distribution: defined on $[0,1]$, used for probabilities and proportions — directly appears in TurboQuant

The Beta distribution in TurboQuant (Lemma 1)

When you take a unit vector in d dimensions and look at one coordinate, that coordinate follows a Beta distribution: $f(x)$ proportional to $(1 - x^2)^{\{(d-3)/2\}}$

This is proved in Lemma 1 of the TurboQuant paper.

As d grows, this Beta distribution converges to $N(0, 1/d)$ — a Gaussian.

Why this matters: once we know the distribution, we can design an optimal quantizer for it. The Lloyd-Max algorithm (Section 4 of this guide) solves for the best codebook.

3.2 Expected value and variance

Expected value is the average outcome of a random variable over many trials. Variance measures how spread out the outcomes are. These two numbers summarize a distribution.

What to learn

- Expected value: $E[X] = \sum_x x \cdot P(X=x)$ for discrete, integral for continuous
- Linearity of expectation: $E[aX + bY] = a \cdot E[X] + b \cdot E[Y]$
- Variance: $\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - E[X]^2$
- Standard deviation: $\sigma = \sqrt{\text{Var}(X)}$
- What 'unbiased estimator' means: $E[\text{estimator}] = \text{true value}$

Why unbiasedness matters in TurboQuant

TurboQuant's inner product algorithm (Algorithm 2) is designed to be an unbiased estimator. This means: if you quantize x and then compute the inner product with y on the compressed version, the expected result equals the true inner product $\langle y, x \rangle$. The MSE-only version (Algorithm 1) does NOT have this property — it has a multiplicative bias of $2/\pi$ at 1-bit width. This is a critical difference.

3.3 The Gaussian distribution

The Gaussian is the most important distribution in statistics and ML. It appears because of the Central Limit Theorem: the average of many independent random variables tends to a Gaussian regardless of their individual distributions.

What to learn

- PDF: $f(x) = (1/\sqrt{2\pi\sigma^2}) \cdot \exp(-(x-\mu)^2 / (2\sigma^2))$
- Parameters: mean μ and standard deviation σ
- Standard normal: $N(0, 1)$ — mean 0, std 1
- The 68-95-99.7 rule: 68% within 1 sigma, 95% within 2 sigma
- Multivariate Gaussian: extends to d -dimensional vectors
- Central Limit Theorem: sums of independent random variables $\rightarrow$ Gaussian

Why Gaussian matters for TurboQuant

After random rotation, each coordinate of the vector follows a Beta distribution. In high dimensions ($d \geq 100$), this Beta distribution is very well approximated by $N(0, 1/d)$. This Gaussian approximation is what lets the paper derive clean theoretical bounds. The Lloyd-Max codebook for a Gaussian is well-studied and can be computed numerically.

3.4 Concentration of measure

In high dimensions, probability mass concentrates in unexpected ways. This is one of the most important and counterintuitive facts in ML. TurboQuant relies on it: in high dimensions, coordinates of a random unit vector become nearly independent, which is what justifies quantizing each coordinate independently.

What to learn

- The curse of dimensionality — intuitions from 2D break in 1536D
- Random unit vectors in high dimensions: almost all pairs are nearly orthogonal
- Near-independence of coordinates in high dimensions
- The Johnson-Lindenstrauss lemma: random projections approximately preserve distances

Exercises for Section 3

#	Exercise
1	Sample 10,000 points from $N(0,1)$. Plot a histogram. Compute the empirical mean and variance.
2	Implement a function to estimate $E[X^2]$ for a given distribution via Monte Carlo sampling.
3	Generate 1000 unit vectors in 100 dimensions. For each pair, compute the dot product. Plot the distribution — what does it look like?
4	Generate a random unit vector in 512 dimensions. Plot the distribution of its coordinates. Compare to $N(0, 1/512)$.
5	Demonstrate the Central Limit Theorem: sample from a uniform distribution, take averages of n samples, show the distribution approaches Gaussian as n grows.
6	Compute the Beta(α , α) PDF for $\alpha = 255$ (corresponding to $d=512$). Overlay with $N(0, 1/512)$. How close are they?

Section 4 Information Theory

The mathematics of compression and communication

Information theory is the science of what can and cannot be communicated or compressed. Claude Shannon (1948) founded the field and proved fundamental limits on compression. TurboQuant's theoretical guarantees are stated using Shannon's distortion-rate function and the Shannon Lower Bound — these appear directly in the paper.

4.1 Entropy

Entropy measures the average information content of a random variable. A fair coin has 1 bit of entropy. A biased coin that always lands heads has 0 bits. The higher the entropy, the more surprised you are on average by the outcome.

What to learn

- Definition: $H(X) = -\sum_x P(x) * \log_2(P(x))$
- Bits as the unit of information
- Uniform distribution maximizes entropy for a given number of outcomes
- Differential entropy: the continuous extension to PDFs
- Entropy of the Gaussian: $h(N(\mu, \sigma^2)) = 0.5 * \log_2(2\pi e \sigma^2)$

4.2 Shannon's source coding theorem

Shannon's theorem tells you the minimum number of bits needed to represent a source. You cannot compress below the entropy. This is the fundamental limit on all compression.

What to learn

- Lossless compression: you can compress to entropy bits per symbol, no further
- Lossy compression: if you allow some error, you can compress further
- The distortion-rate function $D(R)$: the minimum distortion achievable at R bits
- Shannon's lower bound: $D(R) \geq (d / 2\pi e) * 2^{\{2(h(x)-R)/d\}}$
- Why this bound is tight for Gaussians

The Shannon Lower Bound in TurboQuant (Lemma 3)

The paper uses the Shannon Lower Bound to prove that no algorithm can beat:

$$D_{\text{mse}} \geq 4^{-b} \quad (\text{where } b = \text{bits per coordinate})$$

This is Theorem 3 in the paper. TurboQuant achieves:

$$D_{\text{mse}} \leq (\sqrt{3\pi}/2) * 4^{-b} \approx 2.7 * \text{lower_bound}$$

So TurboQuant is within a factor of ~ 2.7 of the theoretical best possible.
No algorithm can do better than 4^{-b} — Shannon's theorem proves this.

4.3 Mutual information

Mutual information measures how much knowing one random variable tells you about another. $I(X;Y) = 0$ means they are independent. $I(X;Y) = H(X)$ means Y tells you everything about X .

What to learn

- Definition: $I(X;Y) = H(X) - H(X|Y)$
- Mutual information as the reduction in uncertainty about X given Y
- $I(X;Y) = 0$ iff X and Y are independent
- Its role in the rate-distortion function: $I(X;Y) \leq R$ bits

4.4 The rate-distortion tradeoff

Rate-distortion theory formalizes the tradeoff between how many bits you use (rate) and how much error you accept (distortion). This is exactly the problem TurboQuant solves — given b bits per coordinate, minimize reconstruction error.

What to learn

- Rate R : the number of bits per symbol used for compression
- Distortion D : the average error between original and reconstructed
- The rate-distortion function $R(D)$: the minimum bits needed to achieve distortion D
- For a Gaussian source with MSE distortion: $D(R) = \sigma^2 * 2^{-2R}$
- This means: each extra bit reduces MSE by a factor of 4 -- 4^{-b}

This is where 4^{-b} comes from

The lower bound in TurboQuant is $D_{\text{mse}} \geq 4^{-b}$.

This comes directly from the Gaussian rate-distortion function $D(R) = \sigma^2 * 2^{-2R}$.

With $R = b$ bits per coordinate: $D = \sigma^2 * 2^{-2b} = \sigma^2 * 4^{-b}$.

For unit-norm vectors, $\sigma^2 = 1/d$ per coordinate, and summing over d coordinates gives 4^{-b} .

Every extra bit of precision reduces MSE by 4x. This is the theoretical ceiling.

Exercises for Section 4

#	Exercise
---	----------

1	Compute the entropy of a fair die (6 outcomes). Then compute for a loaded die ($P(1)=0.5$, rest equal). Which is higher?
2	Compute the differential entropy of $N(0,1)$ using the formula. Verify numerically by sampling and using <code>scipy.stats</code> .
3	For a Gaussian source with $\sigma=1$, plot the rate-distortion curve $D(R) = 4^{-R}$ for $R = 1$ to 8 bits. Mark TurboQuant's upper bound.
4	Given the lower bound $D \geq 4^{-b}$, verify that TurboQuant's empirical MSE (0.36, 0.117, 0.03, 0.009 for $b=1..4$) is within the 2.7x factor claimed.
5	Implement entropy estimation via Monte Carlo: sample from a distribution, estimate $H(X) = -E[\log P(x)]$.

Putting it all together: how these four topics connect

These four topics are not independent — they form a chain of reasoning that runs through all of ML. Here is how they connect:

Topic	In TurboQuant	In ML broadly
Linear algebra	Random rotation matrix Q ; dot product preservation	Weight matrices; attention scores; embeddings
Calculus	Proving distortion bounds via optimization (Eq. 4)	Backpropagation; gradient descent; training
Probability	Beta distribution on rotated coordinates; unbiasedness	Loss functions; sampling; model outputs
Information theory	Shannon lower bound 4^{-b} ; rate-distortion tradeoff	Compression limits; model capacity; entropy

Resources

Linear algebra

3Blue1Brown 'Essence of Linear Algebra' — YouTube series, free, visual, 16 episodes
Gilbert Strang MIT 18.06 — full lecture series on MIT OpenCourseWare, free
Book: 'Linear Algebra and its Applications' by Strang (any edition)
Practical: numpy documentation + implement every operation yourself

Calculus

Khan Academy multivariable calculus — free, structured, well-paced
3Blue1Brown 'Essence of Calculus' — YouTube series, 12 episodes, free
Book: 'Calculus: Early Transcendentals' by Stewart — any edition works

Probability and statistics

Book: 'Introduction to Probability' by Blitzstein & Hwang — free PDF at probabilitybook.net
StatQuest with Josh Starmer — YouTube channel, excellent visualizations, free
scipy.stats — Python library for all common distributions, great for experiments

Information theory

Book: 'Elements of Information Theory' by Cover & Thomas — the standard reference

Chapter 2 is most relevant: entropy, mutual information, source coding
3Blue1Brown 'Information Entropy' video — 20 minutes, excellent intuition builder

Completion checklist

You are ready for Layer 2 when you can do all of the following:

Layer 1 completion criteria

- ☐ You can multiply two matrices by hand and verify with numpy
- ☐ You can explain what an orthogonal matrix does geometrically
- ☐ You can generate a random orthogonal matrix using QR decomposition
- ☐ You can compute a derivative and explain what it means
- ☐ You can implement gradient descent from scratch to minimize a function
- ☐ You can plot a Gaussian distribution and explain mean and variance
- ☐ You can explain what it means for an estimator to be unbiased
- ☐ You can explain the Shannon lower bound and what 4^{-b} comes from
- ☐ You can look at Theorem 1 of the TurboQuant paper and understand what it claims